{Standard Template Library}

C++ — Data Structures — Java — Collections Framework

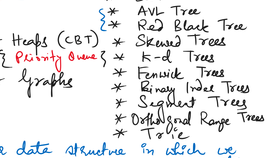
(Recursion)

- **Linear**
  - * Arrays
  - * Linked Lists
  - * Stacks → (LIFO)
  - * Queues → (FIFO)
- **STL** <bits/stdc++.h>
- **Collections** (java.util)
  - * Heaps (CBT) { Priority Queue }
  - * Graphs
- **Non-Linear**
  - * Trees
  - * BST
  - * AVL Tree
  - * RB Black Tree
  - * Skewed Trees
  - * k-d Trees
  - * Fenwick Trees
  - * Binary Index Trees
  - * Segment Trees
  - * Orthogonal Range Trees
  - * Trie

Heap Sort (n log n)

Linked List :→ It is a linear data structure in which we have an entity called node which is connected to other nodes. Depending on the connectivity of nodes, there are three types of linked lists :→

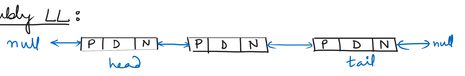
1. Singly Linked List :→ {Each node has some data & pointer to next node.}
2. Doubly Linked List
3. Circular Linked List



We can traverse head to tail because we don't have access to the address/reference/pointer to the current node's previous node.

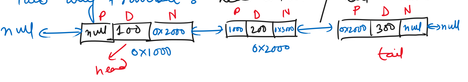
Examples: Browsers :
Spotify Playlist :

Doubly LL :

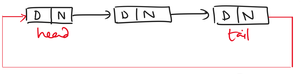


{ Data of the node
Pointer to next node
Pointer to previous node }

* Two way traversal : head to tail / tail to head



* Circular Linked List :→ ⇄ SCLL / DCLL (Cycle)



tail.next = head;

* **Singly Linked List Implementation :→**

```
class Node {
    int data;
    Node next;      {SLL}
    Node prev;      {DLL}
}
```

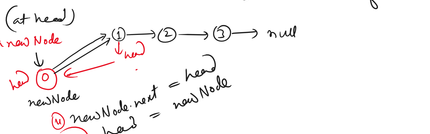
1 → 2 → 3 → null

Most Important Methods:

* Insert
  - → at head / begin
  - → at tail / end
  - → after specific node
* Delete
  - → head
  - → tail
  - → target Value or Node
* Display
  - → all elements of list

Insert : (at head)

① Create a new Node



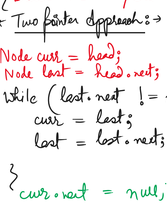
② newNode.next = head

③ head = newNode

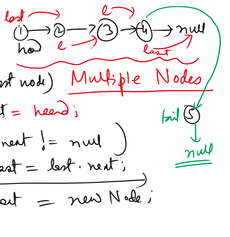
(Insert at tail) :→

Case 1 Empty

(Node → null)
(head)



(Find the last node) **Multiple Nodes**

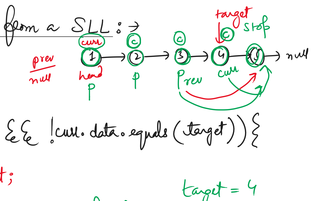


Node last = head;
while (last.next != null)
    last = last.next;
last.next = new Node;

{Delete the last/tail node of a SLL} :→

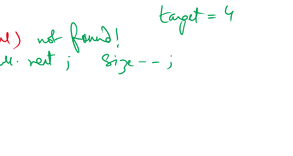
* Two pointer approach :→

Node curr = head;
Node last = head.next;
while (last.next != null) {
    curr = last;
    last = last.next;
}
curr.next = null;



Deleting a target node from a SLL :→

Two pointer approach :→



* Node prev = null;
  Node curr = head;

while (curr != null && !curr.data.equals(target)) {
    prev = curr;
    curr = curr.next;
}

check :→ (curr == null) not found!
prev.next = curr.next; size--;

target = 4